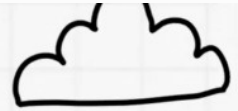


Finite State Machine (FSM)

• Define machine w a finite number of states.

For each (current state + input), the FSM has a definite, predetermined next state / Output

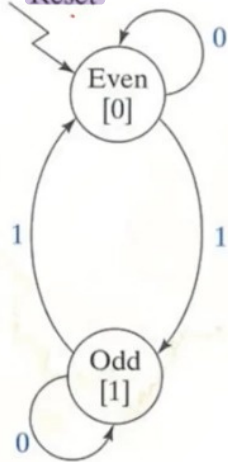
The Parity Checker



The parity checker is one of the simplest machines that checks whether the number of 1's in a logic word is even or odd. If there are odd number of 1's, it will output "1". Otherwise, it will output "0". Let's design the Parity Checker.

1st state

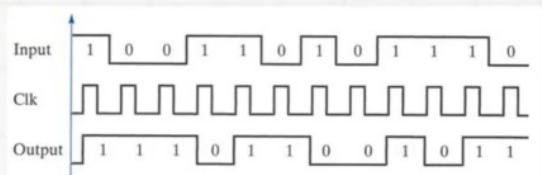
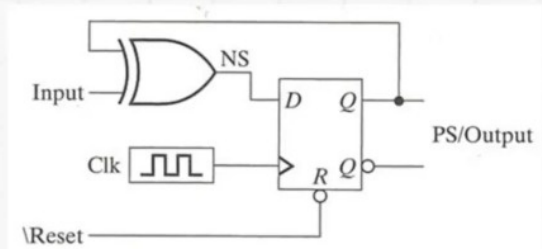
Reset



Present State	Input	Next State	Output
Even	0	Even	0
Even	1	Odd	0
Odd	0	Odd	1
Odd	1	Even	1

Present State	Input	Next State	Output
0	0	0	0
0	1	1	0
1	0	1	1
1	1	0	1

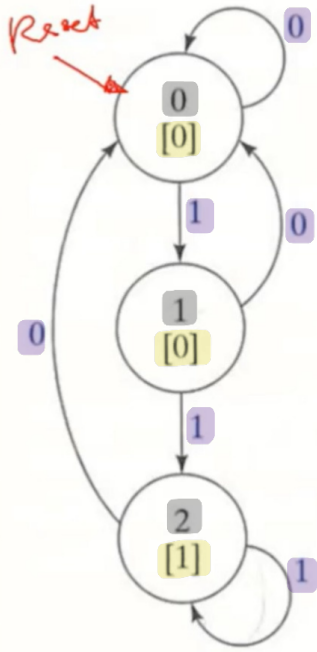
$$\text{NS} = \text{PS} \oplus \text{Input}$$
$$\text{Output} = \text{PS}$$



Moore Machine

- Output depends on current state only [Output waits for state change]
↳ Input only decides what the ^[future current state] next state will be.
∴ Inputs don't instantly show up at output.

state diagram



The way to read the diagram

At state 0, an input of 1 caused the state transition to state 1.

Being in state 1 caused the output [0].

* Notice that the input doesn't directly cause a change in the output.

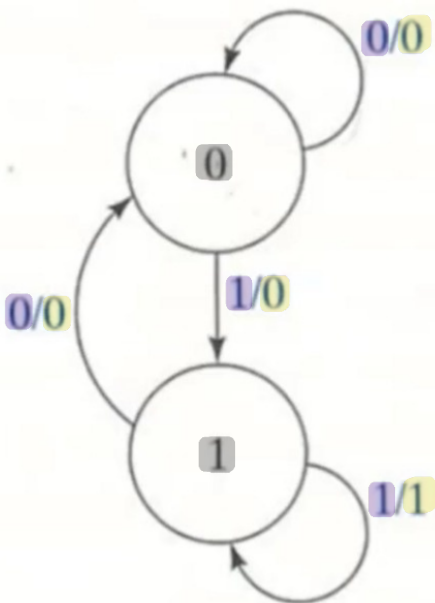
state label

Output

Input

Mealy Machine

- Output depends on current state & input [Output reacts immediately to inputs]
↳ Input changes Output immediately, directly, even if the machine is still in the same state



The way to read the diagram

At state 0, if the input is 1, the machine moves to state 1 and output 0 during that transition

* Notice that the input directly caused a change in output

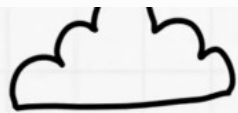
state label

Output

Input

Example 1 [Mealy Machine]

Mealy Machine Example

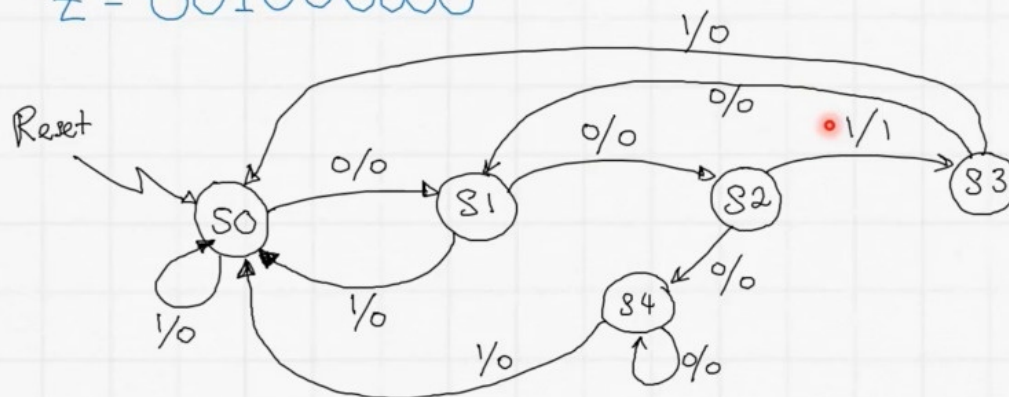


A clocked Mealy sequential circuit with one input (X) and one output (Z) is to be designed. The output is to be 0, unless the input is 0 following a sequence of exactly two 0 inputs followed by a 1 input. Draw the state diagram/graph and the state table.

X = 001110001

Z = 001000000

Legend: state input/output

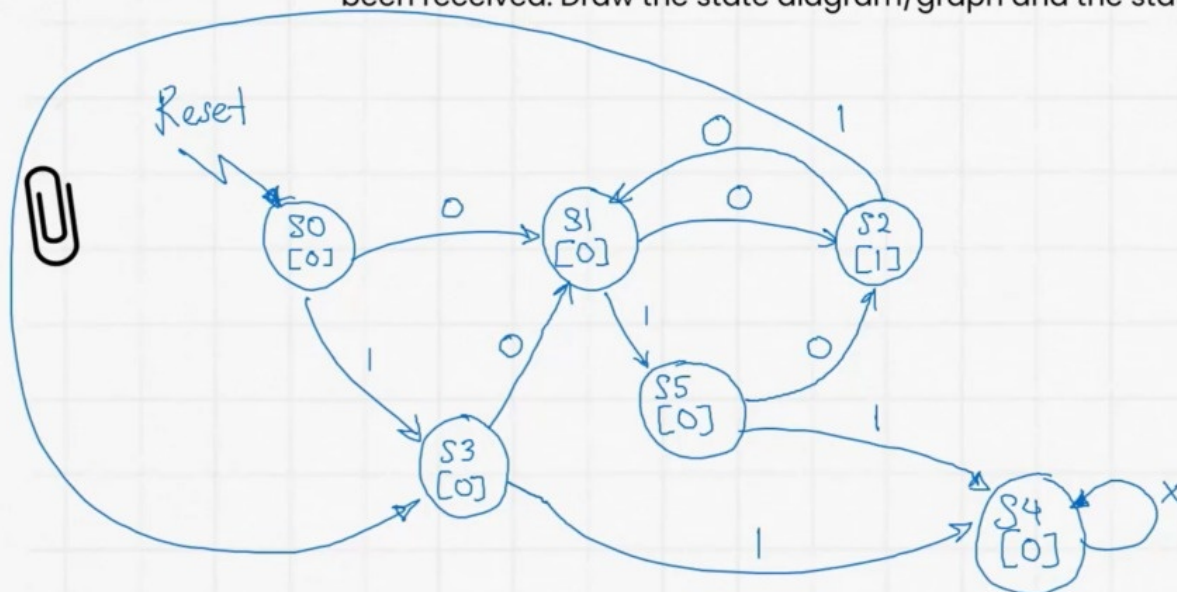


Example 2 [Mealy Machine]

Moore Machine Example



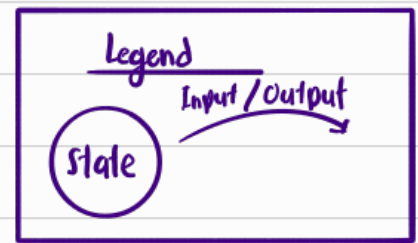
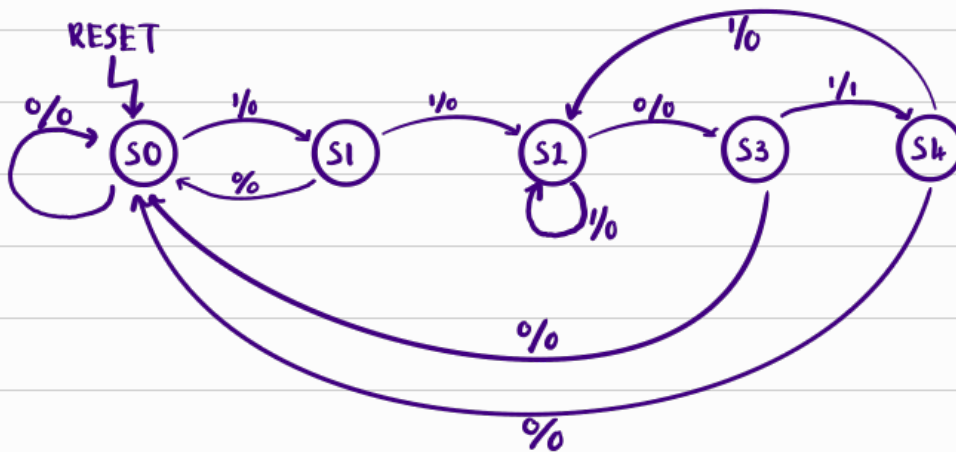
A clocked Moore sequential circuit should have an output of Z = 1 if the total number of 0's received is an even number greater than zero, provided that two consecutive 1's have never been received. Draw the state diagram/graph and the state table.



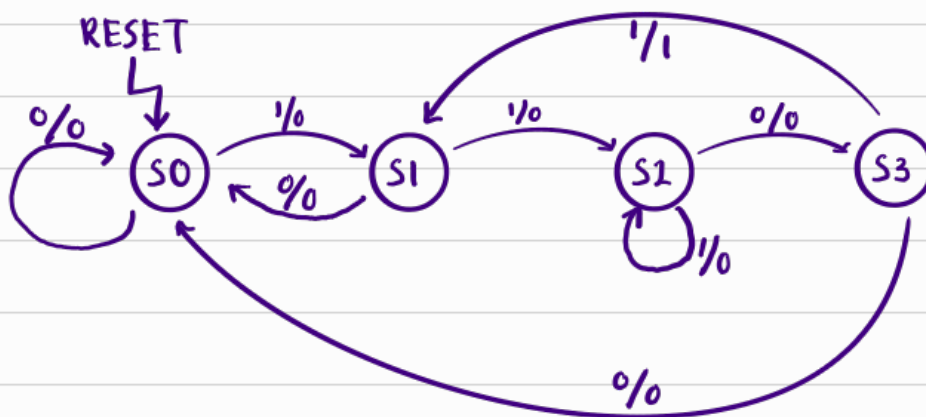
Example 3

Design a Mealy Machine that detects a sequence of inputs, if a sequence of 1101 is detected, Z outputs 1. Otherwise Z outputs 0

Answer



Simplified



State minimisation

As seen in Example 3, both answers are correct, however the simplified one uses less states. There are 2 ways to minimise the states, using a table [Row matching method] or a matrix [Implication table]

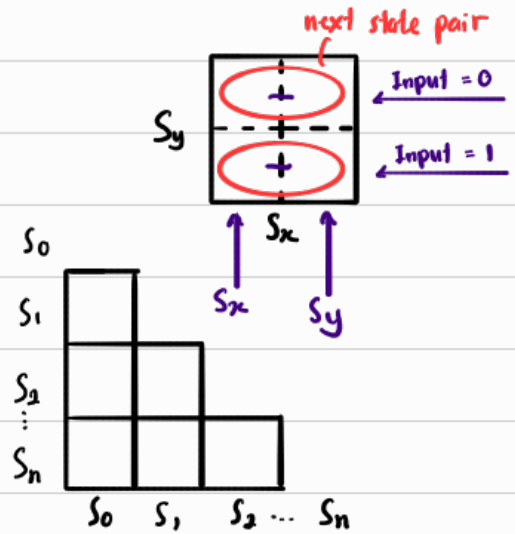
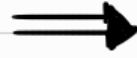
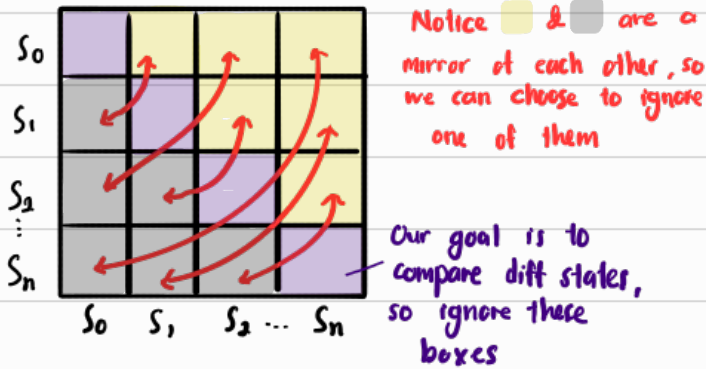
The general idea is just to group all states that gives the same results under the same condition.

Row matching method

Draw state transition table and
combine exactly same rows

For each $\square$, fillw this format

Implication table



Steps

1) Build the table

~ Fill in the next states of all combinations of diff current states

↳ If ...

① Their next states are identical for an input $\rightarrow$ "✓" that input row

↳ If both input are ticked, tick the entire box

② Their outputs are diff $\rightarrow$ "X" that entire box

If both pairs "✓" $\rightarrow$ "✓" box

If either pair "X" $\rightarrow$ "X" box

2) Propagate thru "✓" and "X" through the table

~ Pick any "✓" box [Ex. (S_{10}, S_{12})], then "✓" any other (S_{10}, S_{12}) pair in the table

~ Pick any "X" box [Ex. (S_7, S_{12})], then "X" any box with the pair (S_7, S_{12})

To check if a next state pair can be ✓/X, go to that next state pair box and see if its ✓/X

we can just propagate the "✓" as much as possible, all others will be considered "X"

Generally

We are comparing all pairs of current states to see if

they are mergable. And to see if they are mergable,

we would have to see if their pairs of next state is mergable.

$\therefore$ So see if 2 states are mergable, their [next state's] state have to be mergable also

Mealy

Row matching method

To demonstrate the Row-matching method, let's create a four-bit sequence detector. The detector has a single input X and output Z . The output is asserted after each 4-bit input sequence if it consists of one binary strings 0110 or 1010. The machine returns to the reset state after each and every 4-bit sequence. Let's assume a Mealy implementation.

State transition table

Present State	Next State		Output	
	$X=0$	$X=1$	$X=0$	$X=1$
S_0	S_1	S_2	0	0
S_1	S_3	S_4	0	0
S_2	S_5 S_4	S_6 S_3	0	0
S_3	S_7 S_0	S_8 S_0	0	0
S_4	S_9 S_1	S_{10}	0	0
S_5 S_4	S_{11} S_1	S_{12} S_{10}	0	0
S_6 S_3	S_{13} S_0	S_{14} S_0	0	0
S_7	S_0	S_0	0	0
S_8 S_7	S_0	S_0	0	0
S_9 S_7	S_0	S_0	0	0
S_{10}	S_0	S_0	1	0
S_{11} S_7	S_0	S_0	0	0
S_{12} S_{10}	S_0	S_0	1	0
S_{13} S_7	S_0	S_0	0	0
S_{14} S_7	S_0	S_0	0	0

Optional column

Mealy machine output depends on present state & inputs

Input Sequence	Present State	Next State		Output	
		$X=0$	$X=1$	$X=0$	$X=1$
Reset	S_0	S_1	S_2	0	0
0	S_1	S_3	S_4	0	0
1	S_2	S_4	S_3	0	0
00 or 11	S_3	S_7	S_7	0	0
01 or 10	S_4	S_7	S_{10}	0	0
Not (011 or 101)	S_7	S_0	S_0	0	0
011 or 101	S_{10}	S_0	S_0	1	0

simplified

S_7

S_{10}

S_7

S_7

S_7

S_7

S_7

S_7

S_7

S_7

S_7

S_7

S_7

S_7

S_7

S_7

S_7

S_7

S_7

S_7

S_7

S_7

S_7

S_7

S_7

S_7

S_7

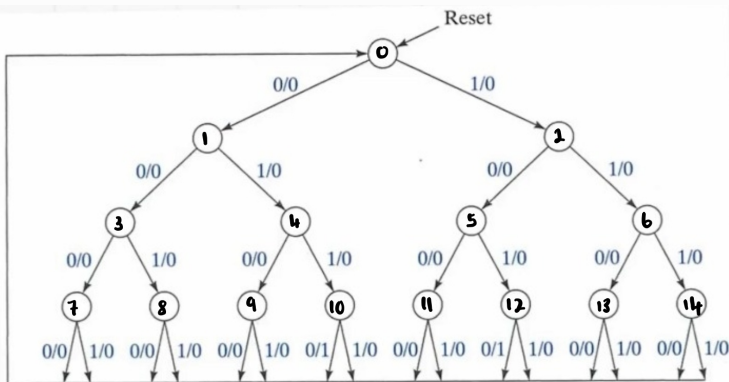
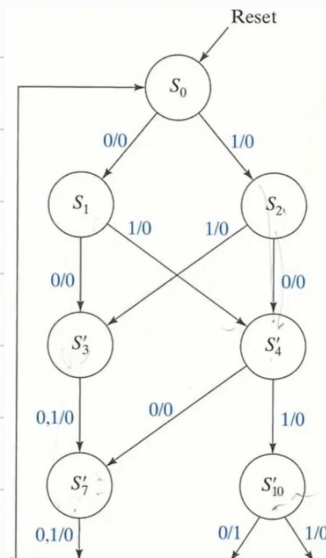
S_7

S_7

S_7

S_7

simplified



Implication table

Beuz of limitation of space, lts start after $s_7, s_8, s_9, s_{11}, s_{12}, s_4 \rightarrow s_7$.

$\therefore$ This means (s_4, s_5) is mergable, (s_3, s_6) is mergable, (s_{10}, s_{12}) is mergable.

s_0	s_1-s_3									
s_1	s_2-s_4									
s_2	s_1-s_5	s_3-s_5								
s_3	s_2-s_6	s_4-s_6								
s_4	s_1-s_7	s_3-s_7	s_5-s_7							
s_5	s_2-s_7	s_4-s_7	s_6-s_7	s_7-s_7						
s_6	s_1-s_7	s_3-s_7	s_5-s_7	s_7-s_7	s_7-s_7					
s_7	s_2-s_7	s_4-s_7	s_6-s_7	s_7-s_7	s_7-s_7	s_7-s_7				
s_8	s_1-s_7	s_3-s_7	s_5-s_7	s_7-s_7	s_7-s_7	s_7-s_7	s_7-s_7			
s_9	s_2-s_7	s_4-s_7	s_6-s_7	s_7-s_7	s_7-s_7	s_7-s_7	s_7-s_7	s_7-s_7		
s_{10}	s_1-s_7	s_3-s_7	s_5-s_7	s_7-s_7	s_7-s_7	s_7-s_7	s_7-s_7	s_7-s_7	s_7-s_7	
s_{11}	s_2-s_7	s_4-s_7	s_6-s_7	s_7-s_7	s_7-s_7	s_7-s_7	s_7-s_7	s_7-s_7	s_7-s_7	s_7-s_7
s_{12}	s_1-s_7	s_3-s_7	s_5-s_7	s_7-s_7	s_7-s_7	s_7-s_7	s_7-s_7	s_7-s_7	s_7-s_7	s_7-s_7

s_3

s_{10}

Present state	Next state		Output	
	$X=0$	$X=1$	$X=0$	$X=1$
s_0	s_1	s_2	0	0
s_1	s_3	s_4	0	0
s_2	s_5	s_6	0	0
s_3	s_7	s_7	0	0
s_4	s_7	s_{10}	0	0
s_5	s_7	s_{12}	0	0
s_6	s_7	s_7	0	0
s_7	s_0	s_0	0	0
s_{10}	s_0	s_0	1	0
s_{12}	s_0	s_0	1	0

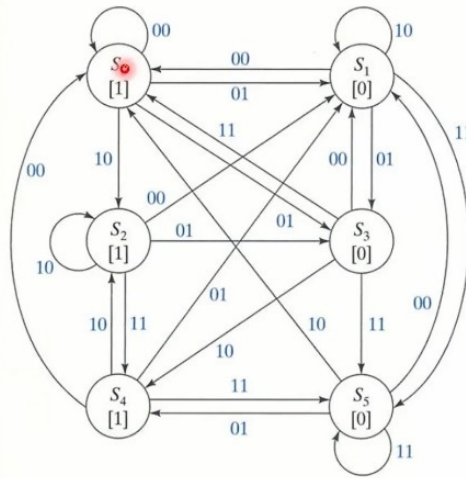
simplified

Present State	Next State		Output	
	$X=0$	$X=1$	$X=0$	$X=1$
s_0	s_1	s_2	0	0
s_1	s_3	s_4	0	0
s_2	s_4	s_3	0	0
s_3	s_7	s_7	0	0
s_4	s_7	s_{10}	0	0
s_7	s_0	s_0	0	0
s_{10}	s_0	s_0	1	0

Moore

[Special Example]

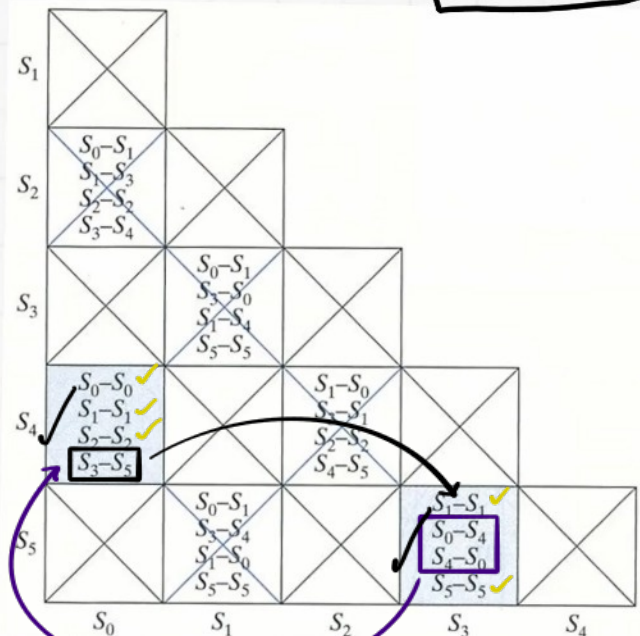
Given the following multiple-input state diagram, let's draw the multiple-input state transition table:



* 2 bit input can have 4 diff combination
∴ Each state must have 4 arrows out of each state

Present State	Next State				Output
	00	01	10	11	
S_0	S_0	S_1	S_2	S_3	1
S_1	S_0	S_3	S_1	S_5	0
S_2	S_1	S_3	S_2	S_4	1
S_3	S_1	S_0	S_4	S_5	0
S_4	S_0	S_1	S_2	S_5	1
S_5	S_1	S_4	S_0	S_5	0

Present State	Next State				Output
	00	01	10	11	
S'_0	S'_0	S_1	S_2	S'_3	1
S_1	S'_0	S'_3	S_1	S'_3	0
S_2	S_1	S'_3	S_2	S'_0	1
S'_3	S_1	S'_0	S'_0	S'_3	0



* In a cycle, the states always share the same future path, so no input can ever distinguish them
∴ Mergable